

YASH PANDEY

12214802722

CSE - Shift - I (2nd Year - 2023-24)

MAIT ROHINI

PSLP - Lab (2023-24) (4th Sem)

(Complete - PSLP - lab File)

CERTIFICATE OF APPRECIATION

THIS CERTIFICATE IS AWARDED TO

FOR SUCCESSFULLY COMPLETION OF
HIS/HER PROJECT WORK.

Affix Your Passport
Size Photo Here

Name of the Student YASH PANDEY

Roll no. 12214802722 Class C-6

Examination Centre MAIT

Date of the Practical Examination _____

Signature

Signature

S.No	Experiment Description	Page No.	Experiment Date	Submission Date	Remarks
<u>1.</u>	Scilab Programs To Perform Addition, Multiplication And Transpose on Matrix Without Function.	2.	06-02-24	13-02-24	A
<u>2.</u>	Program For Fitting Binomial Distribution When n Is Given.	6.	13-02-24	20-02-24	A
<u>3.</u>	WAP To Fitting Binomial Distribution after Computing Mean and Variance.	7.	20-02-24	27-02-24	A
<u>4.</u>	Program for fitting The Poisson's Distribution for Given Value of λ .	9.	27-02-24	05-03-24	A
<u>5.</u>	Program for fitting The Poisson's distribution for Mean Calculation.	10.	05-03-24	12-03-24	A
<u>6.</u>	Program for Plotting The Standard Normal Curve for Given Parameter Values.	11.	12-03-24	19-03-24	A
<u>7.</u>	Write a Program for Regression Lines	12.	16-4-24	23-4-24	A

S.No	Experiment Description	Page No.	Experiment Date	Submission Date	Remarks
8.	Display The First Simplex Table of an LPP	13.	16-4-24	30-4-24	✓
9.	Solve a 3 By 3 Transportation Problem Using MODI Method with Initial Basic feasible Sol ⁿ by North-West Corner Rule	14.	23-4-24	30-4-24	✓
10.	Solve an 3 By 3 Assignment Problem using Single Steps Assignments	15.	23-4-24	30-4-24	✓

Program - 01.

1. Aim → Write Scilab Program To Perform Matrix Addition without Function?

→ clc

disp("Name : Yash Pandey")

disp("Roll NO : 12214802722")

m = input("Enter The N.O. of Rows of Matrix");

n = input("Enter The N.O. of Column of Matrix");

A = zeros(m, n);

B = zeros(m, n);

C = zeros(m, n);

disp("Enter The N.O. of First Matrix Row Wise");

for i = 1:m

for j = 1:n

A(i, j) = input(" ");

end

end

disp("Enter The Number of Second Matrix Row Wise");

for i = 1:m

for j = 1:n

B(i, j) = input(" ");

end

end

for i = 1:m

for j = 1:n

C(i, j) = A(i, j) + B(i, j);

end

end


```
Scilab 6.1.1 Console
File Edit Control Applications ?
Scilab 6.1.1 Console

"Name: Yash Pandey"

"Roll No. : 12214802722"
Enter the no. of rows of matrix3
Enter the no. of coloumns of matrix2

"Enter the number of first matrix row wise"
--> 1
--> 2
--> 3
--> 4
--> 6
--> 8

"Enter the element of second matrix row wise"
--> 9
--> 3
--> 1
--> 5
--> 7
--> 1
```

```
Scilab 6.1.1 Console
File Edit Control Applications ?
Scilab 6.1.1 Console

"Enter the element of second matrix row wise"
--> 9
--> 3
--> 1
--> 5
--> 7
--> 1

"The first matrix Is ::: "

1.    2.
3.    4.
6.    8.

"The second matrix Is ::: "

9.    3.
1.    5.
7.    1.

"The third matrix Is ::: "

10.   5.
4.    9.
13.   9.
```



```
disp (" The First Matrix Is ::: ")
```

```
disp (A)
```

```
disp (" The Second Matrix Is ::: ")
```

```
disp (B)
```

```
disp (" The Third Matrix Is ::: ")
```

```
disp (C)
```

2. Aim → Write Scilab Program To Produce Matrix Multiplication without Fⁿ?

→ `clc`

```
disp (" Name : Yash Bordey ")
```

```
disp (" Roll No : 12214 802722 ")
```

```
m = input (" Enter The N.O. of Rows of 1st Matrix ");
```

```
n = input (" Enter The N.O. of Column of 1st Matrix ");
```

```
p = input (" Enter The N.O. of Rows of 2nd Matrix ");
```

```
q = input (" Enter The N.O. of Column of 2nd Matrix ");
```

```
if n == p
```

```
    disp (" Matrices are Compatible for Multiplication ");
```

```
else
```

```
    disp (" Matrices are Not Compatible for Multiplication ");
```

```
    break;
```

```
end
```

```
A = zeros (m, n);
```

```
B = zeros (p, q);
```

```
C = zeros (m, q);
```



```
disp("Enter The N.O.Of 1st Matrix Row Wise")
```

```
for i = 1:m
```

```
    for j = 1:n
```

```
        A(i,j) = input(" ");
```

```
    end
```

```
end
```

```
disp("Enter The N.O.Of 2nd Matrix Row Wise")
```

```
for i = 1:b
```

```
    for j = 1:2
```

```
        B(i,j) = input(" ");
```

```
    end
```

```
end
```

```
for i = 1:m
```

```
    for j = 1:2
```

```
        for k = 1:n
```

```
            C(i,j) = C(i,j) + (A(i,k) * B(k,j));
```

```
        end
```

```
    end
```

```
end
```

```
disp("The 1st Matrix Is :::")
```

```
disp(A)
```

```
disp("The 2nd Matrix Is :::")
```

```
disp(B)
```

```
disp("The Product of The Matrices Is :::")
```

```
disp(C)
```



```
Scilab 6.1.1 Console
File Edit Control Applications ?
Scilab 6.1.1 Console

    "Name :: Yash Pandey"

    "Roll No. :: 12214802722"
Enter the no. of rows of Ist matrix :: 2

Enter the no. of coloumns of Ist matrix :: 2

Enter the no. of rows of IInd matrix :: 2

Enter the no. of coloumns of IInd matrix :: 2

    "Matrices are comfortable for multiplication"

    "Enter the number of Ist matrix row wise"
--> 2
--> 3
--> 4
--> 5

    "Enter the element of IInd matrix row wise"
--> 6
--> 2
--> 1
--> 4
```

```
Scilab 6.1.1 Console
File Edit Control Applications ?
Scilab 6.1.1 Console

--> 3
--> 4
--> 5

    "Enter the element of IInd matrix row wise"
--> 6
--> 2
--> 1
--> 4

    "The Ist matrix is ->"

    2.    3.
    4.    5.

    "The IInd matrix is ->"

    6.    2.
    1.    4.

    "The product of matrices is ->"

    15.    16.
    29.    28.
```


3. Aim → Write Matlab Program To find Transpose of Matrix Without F^n ?

→ C/C

```
disp(" Name : Yash Pandey ")
```

```
disp(" Roll No : 12214802722 ")
```

```
m = input(" Enter The N.O. of Rows of Matrix ");
```

```
n = input(" Enter The N.O. of Column of Matrix ");
```

```
A = zeros(m,n);
```

```
B = zeros(m,n);
```

```
disp(" Enter The Elements of Matrix Row Wise ")
```

```
for i = 1:m
```

```
    for j = 1:n
```

```
        A(i,j) = input(" ");
```

```
    end
```

```
end
```

```
for i = 1:n
```

```
    for j = 1:m
```

```
        B(i,j) = A(j,i);
```

```
    end
```

```
end
```

```
disp(" The Matrix Is ::: ")
```

```
disp(A)
```

```
disp(" The Transpose Matrix Is ::: ")
```

```
disp(B)
```



```
Scilab 6.1.1 Console
File Edit Control Applications ?
Scilab 6.1.1 Console

    "Name:Yash Pandey"

    "Roll No. : 12214802722"
Enter the no. of rows of matrix3

Enter the no. of coloumns of matrix2

    "Enter the elements of matrix row wise"
--> 1
--> 3
--> 2
--> 5
--> 4
--> 6

    "The matrix is"

    1.   3.
    2.   5.
    4.   6.

    "The transposed matrix is"

    1.   2.   4.
    3.   5.   6.
    0.   0.   0.
```


Experiment - 2.

1. Aim → Program for fitting Binomial Distribution when n is Given?

→ clc;

disp("Name : Yash Pandey");

disp("Roll NO : 122148 02722");

disp("Enter The N.O. of observations");

$n = \text{input}(' \backslash ');$

disp("Value of p ");

$p = \text{input}(' \backslash ');$

disp("Enter The Value of x ");

for $i = 1:n$

$X(1,i) = \text{input}(' \back ');$

end

disp("Enter The N.O. of Frequency");

for $j = 1:n$

$F(1,j) = \text{input}(' \back ');$

end

$EF = \text{sum}(F) \wedge \text{Binomial}(b, n-1);$

disp("Given Frequencies");

disp('F');

disp("Expected Frequency");

disp(EF);

plot 2d3(0:n-1, F);

plot 2d(0:n-1, EF);


```
Scilab 6.1.1 Console
File Edit Control Applications ?
Scilab 6.1.1 Console

    "Name : Yash Pandey"

    "Roll No. 12214802722"

    "enter no of observation"
\10

    "value of p"
\0.3175

    "enter the value of x"
/0
/1
/2
/3
/4
/5
/6
/7
/8
/9
```

```
Scilab 6.1.1 Console
File Edit Control Applications ?
Scilab 6.1.1 Console

/9

    "Enter no of frequency"
/6
/29
/28
/12
/6
/6
/0
/0
/0
/0

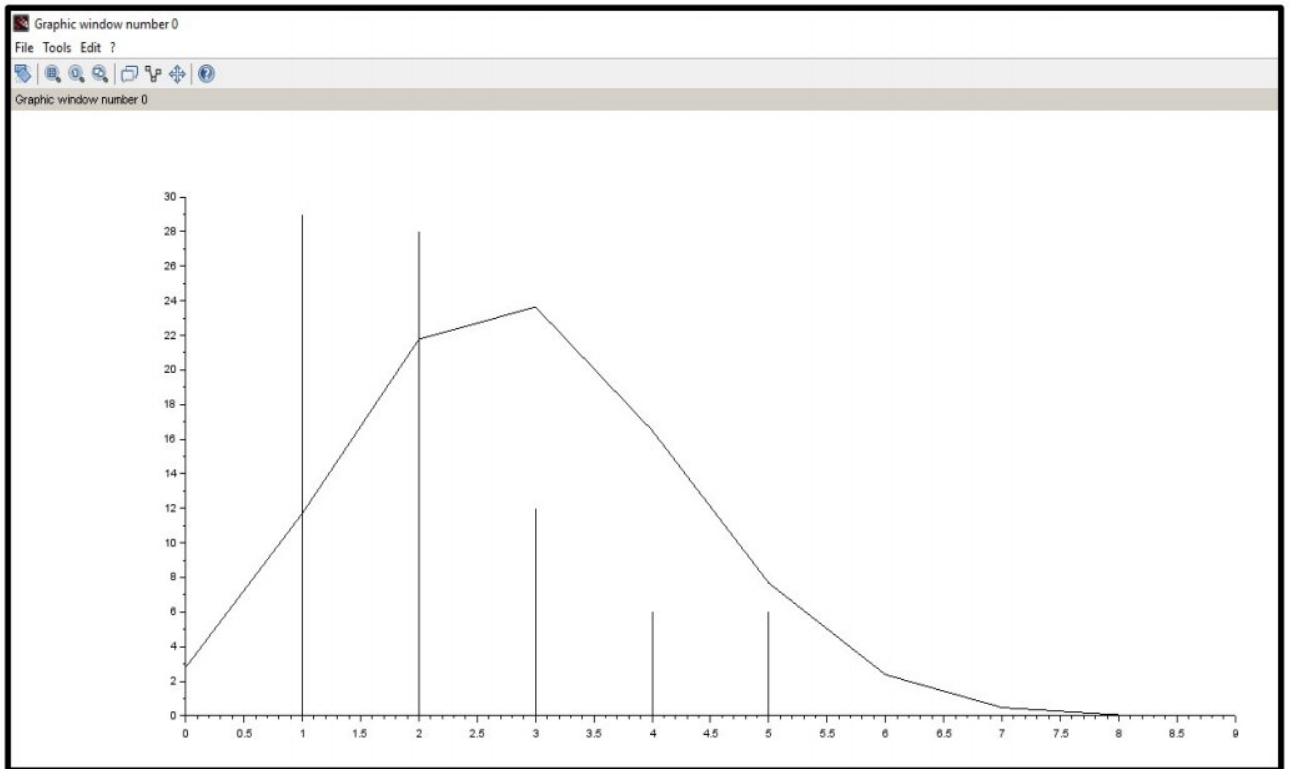
    "given frequencies"

    6. 29. 28. 12. 6. 6. 0. 0. 0. 0.

    "Expected frequencies"

    2.7953948 11.703796 21.778492 23.639901 16.495975 7.6739518 2.3799558 0.4744967 0.0551841 0.0028524

--> |
```

Experiment - 03.

2. Aim $\rightarrow$ WAP To fitting The Binomial Distribution after The Computing of mean and variance?

$\rightarrow$ clc

disp("Name : Yash Pandey")

disp("Roll No : 12214802722")

disp("Enter The N.O. of observations")

n = input(' \')

disp("Enter The Value of X")

for i = 1 : n

 x(1, i) = input(' \')

end

disp("Enter The N.O. of Frequency")

for j = 1 : n

 F(1, j) = input(' \')

end

disp("Mean of The Distribution Is")

MEA = sum(F.^x) / sum(F)

disp(MEA)

p = MEA/n

v = n^p^(1-p)


```
disp("The Volume Is")
```

```
disp(V)
```

```
EF = sum(F) * binomial(p, n-1)
```

```
disp("Given Frequencies")
```

```
disp(F)
```

```
disp("Expected Frequencies")
```

```
disp(EF)
```

```
plot(2d3(0:n-1, F))
```

```
plot(2d(0:n-1, EF))
```



```
Scilab 2023.1.0 Console
File Edit Control Applications ?
Scilab 2023.1.0 Console

    "Name : Yash Pandey"
    "Roll No  ; 12214802722"
    "enter no. of observation"
\5

    "enter the value of X"
\0
\1
\2
\3
\4

    "enter no. of frequency"
\
                                     5
\25
\35
\38
\5

    "enter no. of frequency"
\
                                     5
\25
\35
\38
\5

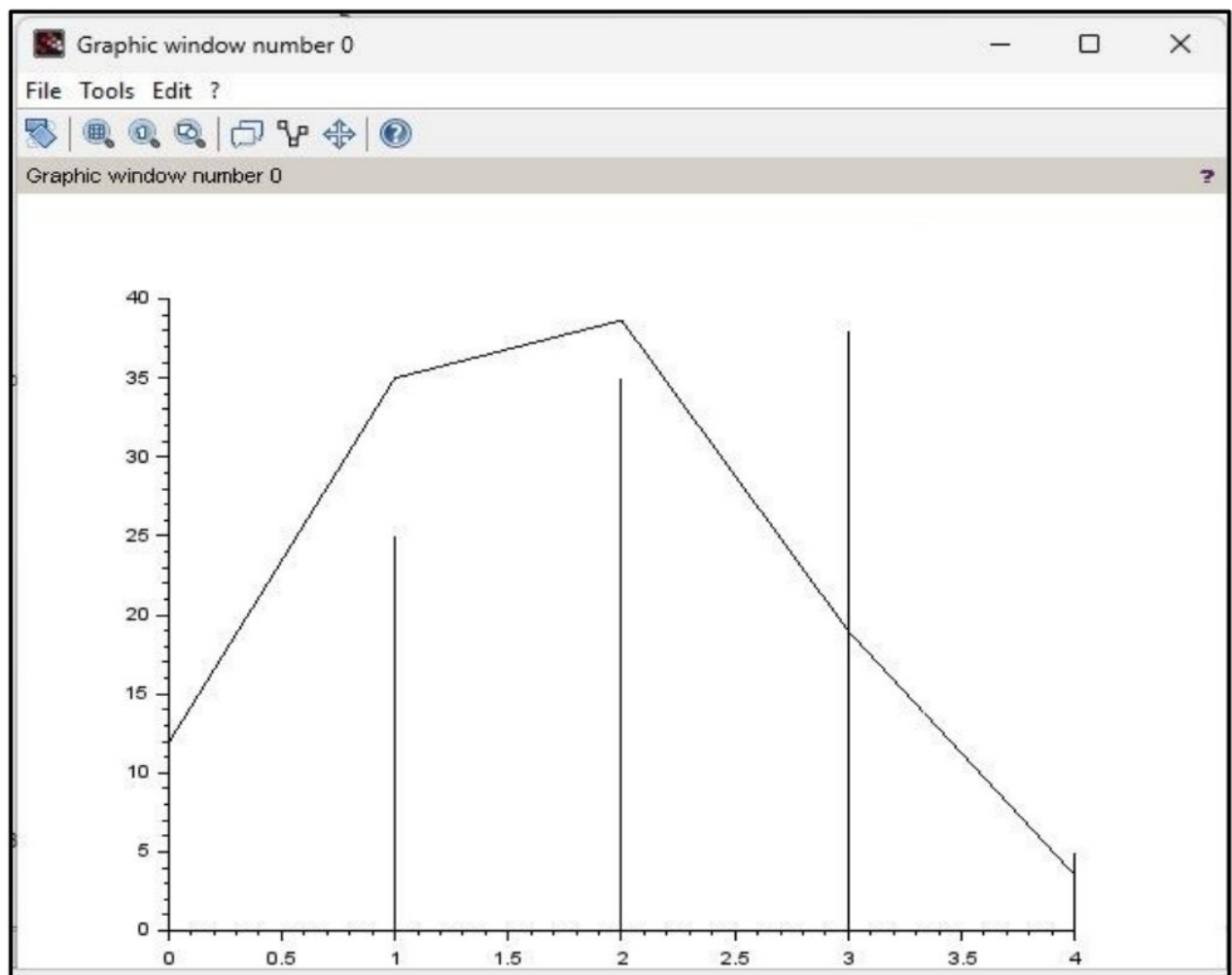
    "Mean of the Distribution is"
    2.1203704

    "The Variance is "
    1.2211763

    "Given Frequencies"
    5.    25.    35.    38.    5.

    "Expected Frequencies"
    11.88202    34.99656    38.653757    18.974727    3.4929361

--> |
```

Experiment - 4.

1. Aim $\rightarrow$ Program for fitting Poisson's Distribution for Given Lambda.

$\rightarrow$ clc

disp (" Name : Yash Pandey ")

disp (" Roll No : 12214802722 ")

disp (" The Mean of The Distribution Is ")

m = input ('\')

disp (" Enter The N.O. of Observations ")

n = input ('/')

disp (" Enter The Value of x ")

for i = 1 : n

 x(1, i) = input ('/')

end

disp (" Enter No of frequency ")

for i = 1 : n

 F(1, i) = input ('/')

end

for i = 1 : n

$$P(1, i) = \text{sum}(F) * \exp(-m) * m^{x(i)} / \text{factorial}(x(i))$$

end

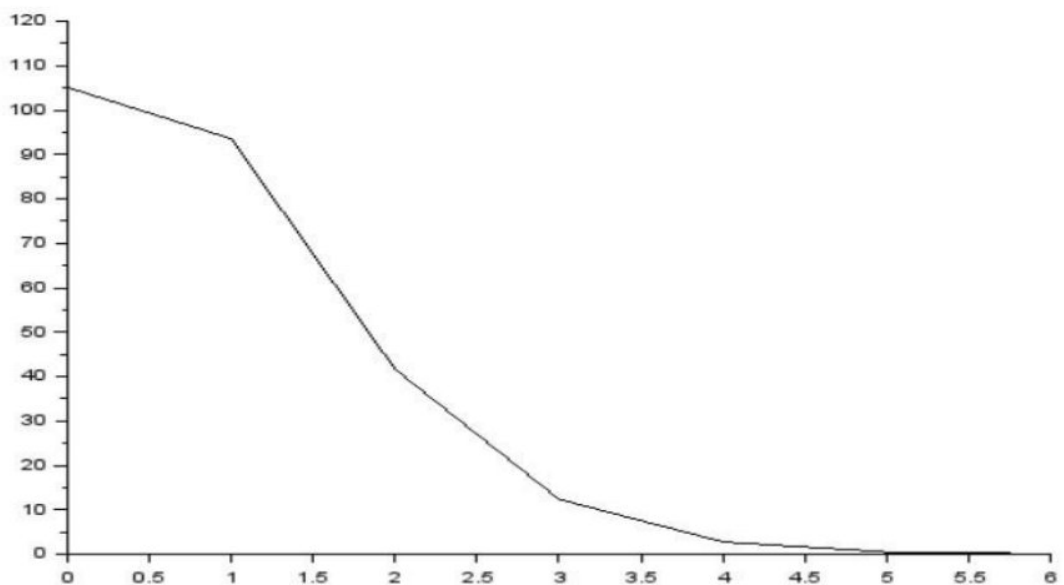
disp (" The Expected frequencies are ")

disp (P)

plot 2d (X, P)


```
"Name : Yash Pandey"  
  
"Roll No : 12214802722"  
  
"Mean of the distribution is"  
/0.89  
  
"Enter The No Of Observations"  
/7  
  
"Enter The Value Of x"  
/0  
/1  
/2  
/3  
/4  
/5  
/6  
  
"enter No of frequency"  
/143  
/50  
/42  
/12  
/5  
/3  
/1  
  
"Expected frequencies are"  
  
105.12787 93.563807 41.635894 12.351982 2.748316 0.4892002 0.0725647
```

Graphic window number 0



Experiment - 5.

1. Aim $\rightarrow$ Program for fitting Poisson's Distribution for Mean Calculation.

$\rightarrow$

clc

disp (" Name : Yash Pandey ")

disp (" Roll No : 12214802722 ")

disp (" Enter N.O. of Observations ")

$n = \text{input}(' \backslash')$

disp (" Enter The Value of x ")

for $i = 1 : n$

$x(1, i) = \text{input}(' \backslash')$

end

disp (" Enter N.O. of Frequency ")

for $j = 1 : n$

$F(1, j) = \text{input}(' \backslash')$

end

disp (" Mean of The Distribution Is ")

$M = \text{sum}(F * X) / \text{sum}(F)$

disp (M)

for $i = 1 : n$

$P(1, i) = \text{sum}(F) * \exp(-M) * M^{\wedge}(x(i)) / \text{factorial}(x(i))$

end

disp (" The Expected Frequencies are ")

disp (P)

plot2d (X, P)

"Name : Yash Pandey"

"Enrollment No: 12214802722"

"Enter no of observation"

\7

"Enter the value of x"

\0

\1

\2

\3

\4

\5

\6

"Enter no. of frequency"

\143

\90

\42

\12

\9

\3

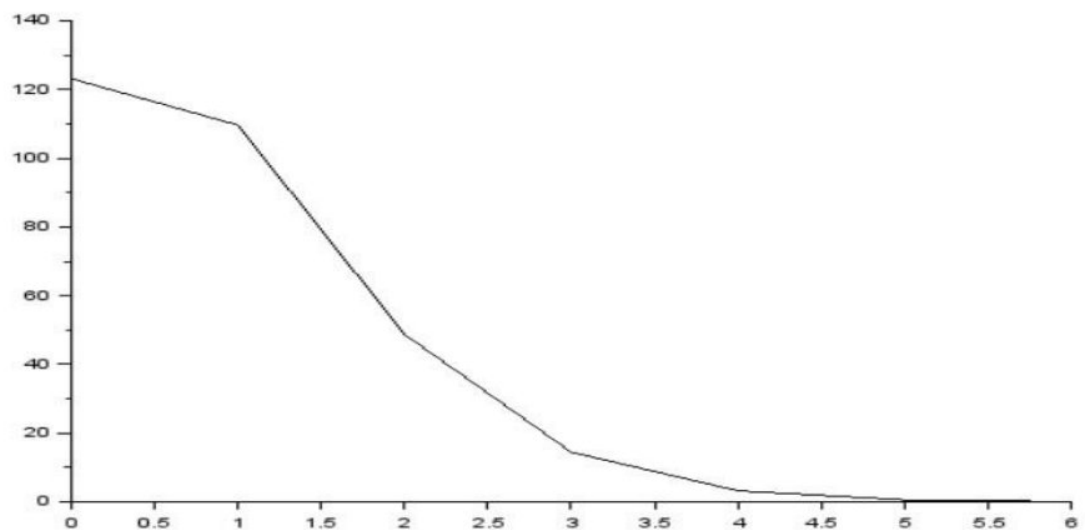
\1

"Mean of the distribution is"

0.89

"Expected Frequencies are"

123.19673 109.64509 48.792063 14.474979 3.2206828 0.5732815 0.0850368



Experiment - 06.

2. Aim $\rightarrow$ Plotting Standard Normal Curve for Given parametric values.

$\rightarrow$ $m_values = [0.5, 1.5, -1.5]$

$s_values = [1, 1.5, 2]$

for $m = m_values$

for $s = s_values$

$t = \text{grid}(1, 100, "net", m, s);$

$x = \text{linspace}((m-4)*s, (m+4)*s, 100);$

$y = (1 / (s * \text{sqrt}(2 * \pi))) * \exp(-(x-m)^2 / (2 * s^2));$

$\text{plot}(x, y);$

$\text{hold on};$

$x.\text{grid}();$

end

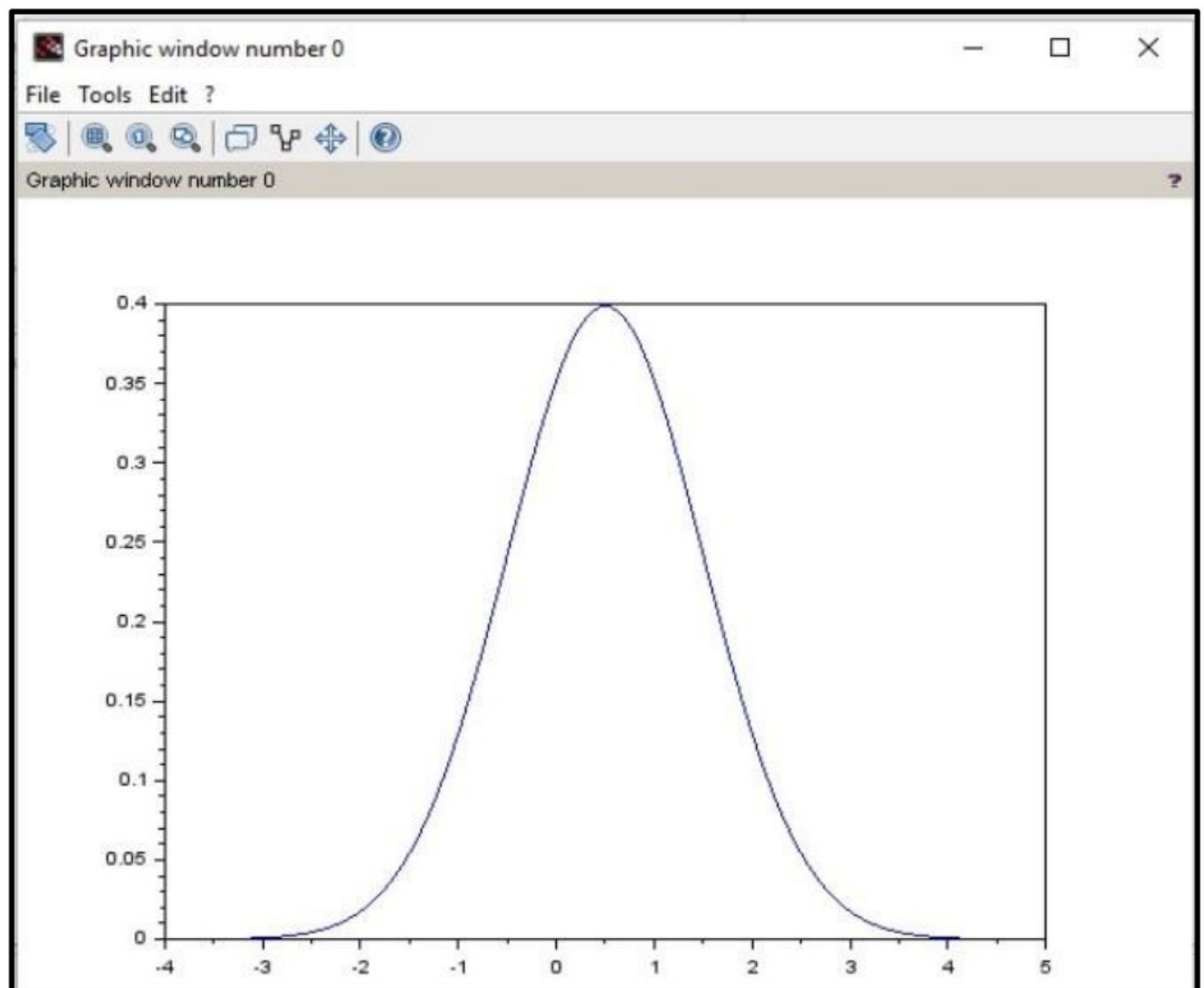
end

$\text{legend}("m=0.5, s=1", "m=1.5, s=1.5", "m=-1.5, s=2");$

$x.\text{label}("x");$

$y.\text{label}("Probability Density Function");$

$\text{title}("Normal Distribution for Various Parametric Values");$



Experiment - 07.

1. Aim $\longrightarrow$ Write a Program for Regression lines.

$\longrightarrow$ C/C++

```
printf ("12214802722 Yash Pandey");
```

```
n = input ("Enter The N.O. of Data Points : ");
```

```
printf ("Enter The Value of xi : ");
```

```
for i = 1 : n
```

```
    x(i) = input (' ');
```

```
end
```

```
printf ("Enter The Value of yi : ");
```

```
for i = 1 : n
```

```
    y(i) = input (' ');
```

```
end
```

```
sumx = 0 ; sumy = 0 ; sumxy = 0 ; sumx2 = 0 ;
```

```
for i = 1 : n
```

```
    sumx = sumx + x(i);
```

```
    sumx2 = sumx2 + (x(i) * x(i));
```

```
    sumy = sumy + y(i);
```

```
    sumxy = sumxy + (x(i) * y(i));
```

```
end
```

```
a = ((sumx2 * sumy - sumx * sumxy) * 1.0 / (n * sumxy - sumx * sumx) * 1.0);
```

```
b = ((n * sumxy - sumx * sumy) * 1.0 / (n * sumx2 - sumx * sumx) * 1.0);
```

```
printf ('The line is Y = %3.3f + %3.3f X', a, b)
```

```
x = (0 : 2 : 20)
```

```
plot (x, a + b * x)
```

Yash Pandey , 12214802722
Enter the number of data points:8

Enter the values of xi

\1

\3

\4

\6

\8

\9

\11

\14

Enter the values of yi

\1

\2

\4

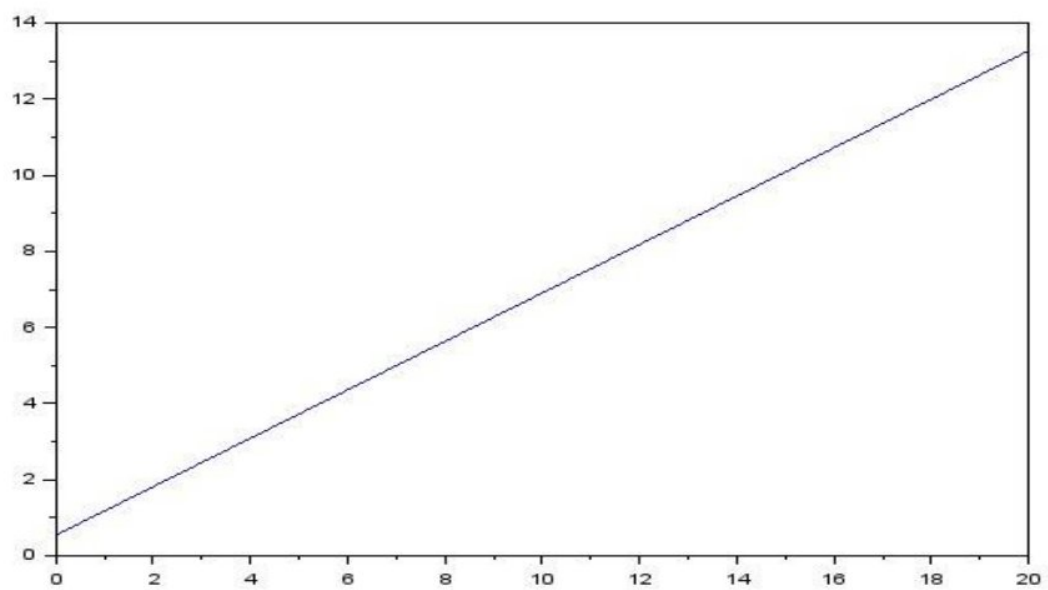
\4

\5

\7

\8

\9



Experiment - 8

```
clc
p=input("Enter number of variables in the objective function :");
q=input("Enter number of constraint equations :");
disp('Enter coefficients of the objective function:')
for i=1:p
    A(i)=input('\n');
end
m=input("Enter 1 to minimize or 2 to maximize");
disp('Enter constraint equations:')
for i=1:q
    disp('enter' + string(i)+' equation: ')
    for j=1:p
        disp('enter coefficient ' + string(j)+':')
        B(i,j)=input('\n');
        if(j==p) then
            printf("\n whether you want to maximize or minimize this equation:\nEnter \n 1. for <= \n 2,
for >=")
            C(i)=input('\n');
            printf("enter constant");
            D(i)=input('\n');
        end
    end
end
printf("\n\n The LPP is :::\n\n\n");
if(m==1) then
    printf("MIN \n");
else
    printf("MAX \n");
end
for i=1:p
    if(i==p) then
        printf("%d%d", A(i),i);
    else
        printf("%dx%d + ",A(i),i);
    end
end
printf("\nConstraint equations are \n");
for i=1:q
    for j=1:p
        printf("%dx%d + ",B(i,j),j);
        if(j==p) then
            if(C(i)==1) then
                printf(" <= %d",D(i));
            else if(C(i)) then
                printf(" >= %d",D(i));
            end
        end
    end
end
end
```

```

end
printf("\n");
end

printf("\n\n ===== IS SIMPLEX TABLE ===== \n\n\n");
printf(" CJ ");
for i=1:p-1
    printf(" %d", A(i));
end
for i=1:q-1
    printf(" 0s%d", i);
end
printf("\n----- ");
printf("\nBV Cb Xb [");
for i=1:p
    printf(" x%d", i);
end
for i=1:q
    printf(" s%d", i);
end
printf(" Min Xb/x");
printf("\n-----\n")
for i=1:p
    printf("s%d 0 %d ", i, D(i));
    for j=1:q
        printf("%d ", B(i,j));
    end
    for j=1:q
        if(j==i) then
            printf("1 ");
        else
            printf("0");
        end
    end
    printf("\n");
end
printf("-----");
printf("\n ZJ-CJ [");
for i=1:p
    printf(" -%d", A(i));
end
for i=1:q
    printf(" 0");
end
end

```


Enter number of variables in the objective function :2

Enter number of constraint equations :2

"Enter coefficients of the objective function:"

\2

\3

Enter 1 to minimize or 2 to maximize2

"Enter constraint equations:"

"enter1 equation: "

"enter coefficient 1:"

\2

"enter coefficient 2:"

\5

whether you want to maximize or minimize this equation:

Enter

1. for <=

2, for >=

\1

enter constant

\9

"enter2 equation: "

"enter coefficient 1:"

\1

"enter coefficient 2:"

\3

whether you want to maximize or minimize this equation:

Enter

1. for <=

2, for >=

\1

enter constant

\10

The LPP is ::::

MAX

$$2x_1 + 3x_2$$

Constraint equations are

$$2x_1 + 5x_2 + s_1 \leq 9$$

$$1x_1 + 3x_2 + s_2 \leq 10$$

===== IS SIMPLEX TABLE =====

CJ 2 0 s1

BV Cb Xb [x1 x2 s1 s2 Min Xb/x

s1 0 9 2 5 1 0

s2 0 10 1 3 0 1

ZJ-CJ [-2 -3 0 0

-->

Experiment – 9

```
clc

prices = [1, 2, 3; 8,5, 4; 3, 1, 6]
demand = [100, 30, 70]
supply = [110, 40, 50]
prices = evstr(x_matrix('setprices', prices));
demand = evstr(x_matrix('setdemand', demand));
supply = evstr(x_matrix('submit offer', supply));
LEFT = 1
RIGHT = 2
UP = 3
DOWN = 4

// The function of calculating the cost of the transferred plan
function res=cost(prices, plan)
    cntCols = length(prices(1,:))
    cntRows = length(prices(:,1))
    res = 0
    for i=1:cntRows
        for j=1:cntCols
            res = res + prices(i,j) * plan(i,j)
        end
    end
endfunction

// a function that looks for available angles in a given direction
// and returns them in descending order of proximity to the edge
function [corners, success] = getAvailableCorner(basis, direction, initialPoint, i, j)
    success = 0
    corners = []
    currentCorner = 1
    cntCols = length(basis(1,:))
```

```

cntRows = length(basis(:,1))
colModifier = 0;
rowModifier = 0;
if direction == LEFT then
    colModifier = -1
end
if direction == RIGHT then
    colModifier = 1
end
if direction == UP then
    rowModifier = -1
end
if direction == DOWN then
    rowModifier = 1
end
i = i + rowModifier
j = j + colModifier
while i ~= 0 && j ~= 0 && i <= cntRows && j <= cntCols
    if basis(i,j) ~= 0 || [i, j] == initialPoint then
        corners(currentCorner,:) = [i,j]
        currentCorner = currentCorner + 1
        success = 1
    end
    i=i+rowModifier
    j=j+colModifier
end
if success == 1 then
    cornersReverse=[]
    for iter=1:length(corners(:,1))
        cornersReverse(iter,:) = corners(length(corners(:,1)) - iter + 1, :)
    end
    corners = cornersReverse

```



```

end
endfunction
// recursive looping function
function [nodes, success] = buildCycle(basis, initialPoint, currentPoint, direction)
    success = 0
    nodes = []
    possibleDirections = []
    if initialPoint == currentPoint then
        possibleDirections = [LEFT, RIGHT, UP, DOWN]
    else if direction == LEFT || direction == RIGHT then
        possibleDirections = [UP, DOWN]
    else if direction == UP || direction == DOWN then
        possibleDirections = [LEFT, RIGHT]
    end; end; end
    for directionIdx= 1:length(possibleDirections)
        [corners, suc] = getAvailableCorner(basis, possibleDirections(directionIdx), initialPoint,
currentPoint(1), currentPoint(2))
        if suc == 1 then
            possibleToCloseCycle = 0
            successWithCorners = 0
            for cornIdx= 1:length(corners(:,1))
                if (corners(cornIdx,:) == initialPoint) then possibleToCloseCycle = 1
                    continue
                end
                [subNodes, suc] = buildCycle(basis, initialPoint, corners(cornIdx,:),
possibleDirections(directionIdx))
                if suc == 1 then
                    successWithCorners = 1
                    nodeIdX = 1
                    nodes(nodeIdx, :) = currentPoint
                    for subNodeIdx = 1:length(subNodes(:,1))
                        nodeIdX = nodeIdX + 1

```

```

        nodes(nodeIdx, :) = subNodes(subNodeIdx,:);
    end
break
end
end
if successWithCorners == 1 then
    success = 1
    break
else if possibleToCloseCycle == 1 then
    nodes(1,:) = currentPoint
    nodes(2,:) = initialPoint
    success = 1
    break
    end;
end
end
end
endfunction
cntCols = length(prices(1,:))
cntRows = length(prices(:,1))
plan = [] // reference plan
plan(cntRows, cntCols) = 0 // fill with zeros
// Calculation of the original reference plan using the northwest corner method
tempDemand = demand
tempSupply = supply
for j=1:cntCols // iterate over columns (customers)
    for i=1:cntRows // iterate over rows (suppliers)
        currentSupply = min(tempDemand(j), tempSupply(i))
        plan(i,j) = currentSupply
        tempDemand(j) = tempDemand(j) - currentSupply
        tempSupply(i) = tempSupply(i) - currentSupply
        if tempDemand(j) == 0 then

```



```

        break
    end
end
end
disp("Initial plan:")
disp(plan)
printf("\nThe cost is %d Nf.Đµ.\n\n", cost(prices, plan))
// Plan optimization
optimal = 0
UNKNOWN_POTENCIAL=9999999
iteration = 0
while optimal == 1
    iteration = iteration + 1
    potencialU = []
    potencialV = []
    for i=1:cntRows
        potencialU(i) = UNKNOWN_POTENCIAL // type unknown yet potential
    end
    for i= 1:cntCols
        potencialV(i) = UNKNOWN_POTENCIAL
    end
    potencialU(1)=0
    continue
    Potentialing = 1
    // calculation of potentials by points in the route
    while continuePotentialing == 1
        continue
        Potentialing = 0
    // we continue to calculate the potentials if
    // for one of the values 'â€œâ€œ'of the plan, both potentials are unknown
    for j=1:entCols // iterate over columns (customers)
        for i=1:cntRows // iterate over rows (suppliers)

```

```

    if (plan(i,j) == 0) then
        continue
    end

    if potencialU(i) == UNKNOWN_POTENCIAL && potencialV(j) -
UNKNOWN_POTENCIAL then
        continuePotencialing = 1
        continue
    end

    if potencialU(i) == UNKNOWN_POTENCIAL then
        potencialU(i) = prices(i,j) - potencialV(J)
    end

    if potencialV(j) == UNKNOWN_POTENCIAL then
        potencialV(j) = prices(i,j) - potencialU(i)
    end

end
end
end

// Calculating estimates for non-basic variables
notBasis = []// reference plan
notBasis(cntRows, cntCols) = 0//fill with zeros
optimal = 1
maxI = 0;
maxJ = 0;
maxNB = 0;
for j=1:cntCols // iterate over columns (customers)
    for i= 1:cntRows // iterate over rows (suppliers)
        if (plan(i,j)==0) then
            continue
        end
        notBasis(ij) = potencialU(i) + potencialV(j) - prices(i,j)
        if notBasis(i,j) > 0 then
            optimal = 0

```



```

        if maxNB < notBasis(i,j) then
            maxNB = notBasis(i,j)
            maxI = i
            maxJ = j
        end
    end
end
end

if optimal == 1 then
    printf("Iteration %d. The current plan is optimal!", iteration)
    break
else
    printf("Iteration %d. Current plan is not optimal. Plan optimization", iteration)
end

[nodes, success]=buildCycle(plan, [maxI, maxJ], [maxI, maxJ], "")
if success == 0 then
    disp("Loop building error. Shutdown")
    break
end

// Among the even nodes of the cycle (those who will have a negative 0) looking for the
minimum value
minNode=999999999
for node = 2:2:length(nodes(:,1))
    if minNode> plan(nodes(node, 1), nodes(node, 2)) then
        minNode = plan(nodes(node, 1), nodes(node, 2))
    end
end

for node = 2:length(nodes(:,1))
    nodeI = nodes(node, 1)
    nodeJ = nodes(node, 2)

```

```

    if modulo(node, 2) - 0 then
        plan(nodeI, nodeJ) = plan(nodeI, nodeJ) - minNode // for even subtract min, meaning
    else
        plan(nodeI, nodeJ) = plan(nodeI, nodeJ) + minNode // for odd ones add min, meaning
    end
end
disp("New plan:")
disp(plan)
printf("\nThe cost is %d N/Đμ.\n\n", cost(prices, plan))
end

tableStr = 2;
table = []
table(1,:) = [" " "From supplier" "To the consumer" "Quantity"];

for i=1:cntRows
    for j = 1:cntCols
        if plan(i,j) ~= 0 then
            str = []
            str(1)=" "
            str(2:4)=string([i, j, plan(i,j)])
            table(tableStr,:)= str
            tableStr=tableStr + 1
        end
    end
end
disp(table)
f=createWindow();
f.figure_size = [400 400];
f.figure_name = "Final answer";
as=f.axes_size;
ut=uicontrol("style", "table",...

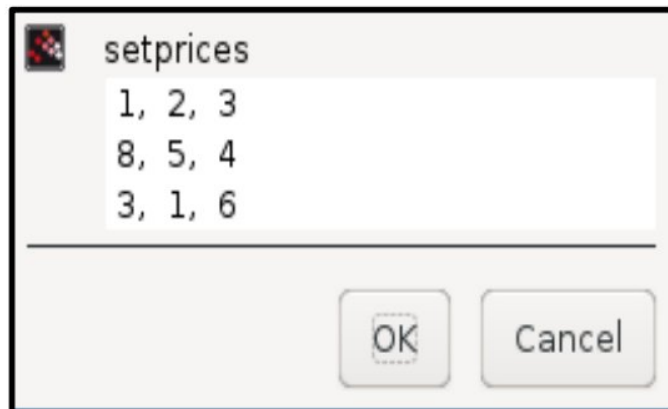
```


"string", table,...

"position", [0 -50 400 400],...

"tag", "Final answer");

matrix(ut.string, size(table))



Scilab Input Value Request

 setdemand

100, 30, 70

OK Cancel

Scilab Input Value Request

 submit offer

110, 40, 50

OK Cancel

	From supplier	To the consumer	Quantity
	1	1	100
	1	2	10
	2	2	20
	2	3	20
	3	3	50

"Initial plan:"

```
100.  10.  0.
0.    20.  20.
0.    0.   50.
```

The cost is 600 Nf.Đμ.

```
" " "From supplier" "To the consumer" "Quantity"
" " "1"           "1"           "100"
" " "1"           "2"           "10"
" " "2"           "2"           "20"
" " "2"           "3"           "20"
" " "3"           "3"           "50"
```


Experiment – 10

```
clc
n=input("how many workers and job do you have :");
printf("\nenter time of\n\n");
for i=1:n
    for j=1:n
        printf("worker %d job%d:",i,j);
        T(i,j) = input("\n");
    end
end
printf("\nDATA YOU ENTERED IS :\n ");
for i=1:n
    printf("job%d ",i);
end
for i=1:n
    printf("\nworker%d ",i);
    for j=1:n
        printf("%d ",T(i,j));
    end
end
minim=[1000,1000,1000,1000,1000,1000,1000,1000,1000];
for i=1:n
    for j=1:n
        if(T(i,j) <= minim(i)) then
            minim(i)=T(i,j);
        end
    end
end
printf("\n");
for i=1:n
    for j=1:n
        T(i,j)=T(i,j)-minim(i);
    end
end
printf("\n");
printf("\n\n*****data after row minimum decrement is*****\n\t\t");
for i=1:n
    printf("job%d ",i);
end
for i=1:n
    printf("\nworker%d ",i);
    for j=1:n
        printf("%d ",T(i,j));
    end
end
zerr=[1000,1000,1000,1000,1000,1000];
zerc=[1000,1000,1000,1000,1000,1000];
for i=1:n
    for j=1:n
        if(T(i,j)==0) then
```

```

        zerr(i)=0;
        zerc(j)=0;
    end
end
end
f=0;
y=0;
for i=1:n
    if(zerr(i)) then
        f=1;
    end
end
for i=1:n
    if(zerc(i)) then
        y = 1
    end
end
if((f==1)||(y==1)) then
    mn=[1000,1000,1000,1000,1000,1000,1000,1000,1000];
    for j=1:n
        for i=1:n
            if(T(i,j) <= mn(i)) then
                mn(j)=T(i,j);
            end
        end
    end
    for j=1:n
        for i=1:n
            T(i,j)=T(i,j)-mn(j);
        end
    end
    printf("\n\n*****data after column minimum decrement is*****");
    for i=1:n
        printf("job%d ",i);
    end
    for i=1:n
        printf("\nworker%d ",i);
        for j=1:n
            printf("%d ",T(i,j));
        end
    end
end
printf("\n\n\n*****Final job assignment is*****\n");
for i=1:n
    for j=1:n
        if(T(i,j)==0) then
            printf("in assign job %d to worker %d\n",j,i);
            for z=1:n
                if (z==i) then
                    continue;
                end
                if(T(z,j)== 0) then

```

```

        T(z,j)=1000,
    end
    for l=1:n
        if(l==j) then
            continue;
        end
        if (T(i,l)==0) then
            T(i,l)=1000;
        end
    end
end
end
end
end
printf("\n");

```


how many workers and job do you have :3

enter time of

worker 1 job1:
\2

worker 1 job2:
\4

worker 1 job3:
\5

worker 2 job1:
\6

worker 2 job2:
\3

worker 2 job3:
\2

worker 3 job1:
\5

worker 3 job2:
\4

worker 3 job3:
\6

DATA YOU ENTERED IS :

job1 job2 job3

worker1 2 4 5

worker2 6 3 2

worker3 5 4 6

*****data after row minimum decrement is*****
job1 job2 job3

worker1 0 2 3

worker2 4 1 0

worker3 1 0 2

*****Final job assignment is*****

in assign job 1 to worker 1

in assign job 3 to worker 2

in assign job 2 to worker 3